100 Days of Linux - Day 065

Title: Command-Line Arguments in Bash

Today's Goal
Learn how to pass values to a Bash script using command-line arguments.

Why This Matters
Most real-world shell scripts accept arguments so they can work with different files, users, and directories without editing the script.

Concepts of the Day

Special Variables:
$1 = First argument
$2 = Second argument
$3 = Third argument
$# = Number of arguments
$@ = All arguments

Example Script:
```
#!/bin/bash
echo "First: $1"
echo "Second: $2"
echo "Total: $#"
```

Run:
```
bash args.sh Vishal Linux
```

Beginner Lesson
Instead of asking for input with **read**, you can pass values directly when running the script. This makes scripts faster, reusable, and ideal for automation.

Practice Questions

1. Create a script named args.sh that prints the first and second command-line arguments.
2. Run args.sh with your name and favorite Linux distribution as arguments.
3. Modify the script to display the total number of arguments using $#.
4. Modify the script to display all arguments using $@.
5. Display your current working directory after running the script.

Hints

Remember: arguments are passed after the script name, for example: bash args.sh Vishal Ubuntu.

Mini Challenge

Create student.sh that accepts Name, Course, and City as command-line arguments and prints a formatted student profile along with the total number of arguments.

Common Mistakes
Trying to use $1 without passing any arguments, or confusing command-line arguments with variables created using read.

Did You Know?
Many Linux utilities such as cp, mv, grep, and tar rely entirely on command-line arguments.

Pro Tip
Use command-line arguments for automation and use read only when interactive user input is required.

Answer Key (Instructor Only)
Q1: echo $1 && echo $2
Q2: bash args.sh Vishal Ubuntu
Q3: echo $#
Q4: echo $@
Q5: pwd

Homework
Create three Bash scripts that accept different arguments, such as filenames, usernames, and project names.

Next Day Preview
Tomorrow you'll learn conditional statements using if, else, and elif to make Bash scripts capable of making decisions.